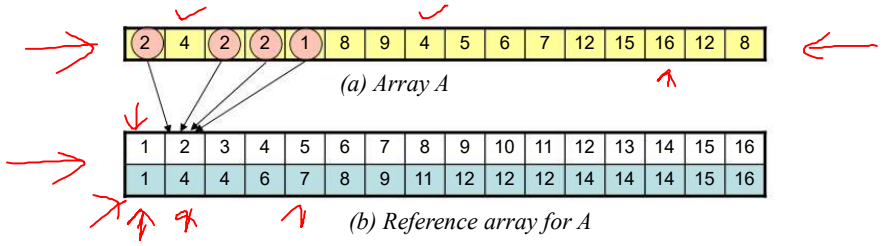
Non-Comparison Sorting

1

Counting Sort

Reference Array

The counting sort procedure sorts an array A[1..n] of integers in an ascending order. An auxiliary array C[1..n], called reference array is used. It stores ***count*** of all elements in A that are ***less than or equal to the corresponding index of reference array.*** Figure (a) shows a sample array and figure (b) shows the corresponding reference array



2	4	2	2	1	8	9	4	5	6	7	12	15	16	12	8

(a) Array A

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	4	4	6	7	8	9	11	12	12	12	14	14	14	15	16

(b) Reference array for A

For example, there are four elements {1,2,2,1}in array A, which are ***less than or equal to index 2 of the reference table***

2

Counting Sort

Reference Array

In order sort a given array, a reference array is created which holds count of elements equal to or less than the corresponding index. Counting is performed in two steps, as illustrated below.

1	2	3	4	5	6	7	8	9	10	11	12	13	15	15	16
5	4	2	2	1	8	9	4	2	6	7	12	15	16	12	8

(a) Source Array

Step #1:*Count elements in source array that are equal to array indexes*

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	3	0	2	1	1	1	2	1	0	0	2	0	0	1	1

(b) Intermediate Array

Step #2: *Count elements that less than or equal to array indexes*

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	4	4	6	7	8	9	11	12	12	12	14	14	14	15	16

(c) Reference Array

There is ***one element*** less than or equal to index 1, ***four elements*** less than or equal to index 2 ***four elements*** less than or equal to index 3, ***six elements less*** than or equal to index 4,and so forth.

3

Counting Sort

Algorithm

In order to sort the array A[1..n], ***two auxiliary arrays, B and C*** are used. The array ***B*** is the target array which stores the ***sorted integers*** The array ***C*** is the ***reference array*** The sorting algorithm works by as follows:

*Step #1:Three indexes **j** ,**i** and **k** are used to access the array **A, B, C** respectively*
*The array **A** is scanned from **right-to-left using j. j** is initially set to **n***

*Step #2: (i) The index **i** is set to **A[j], i.e** **i=A[j]***
*(ii) Index **k** is set to **C[i]** **i.e,** **k=C[i]***
*(iii) The element **A[j]** in the source array is copied to **B[k] i.e B[k]=A[j]***
*(iv) The count **C[i]** of reference array is reduced by 1 i.e **C[i]=C[i]-1***

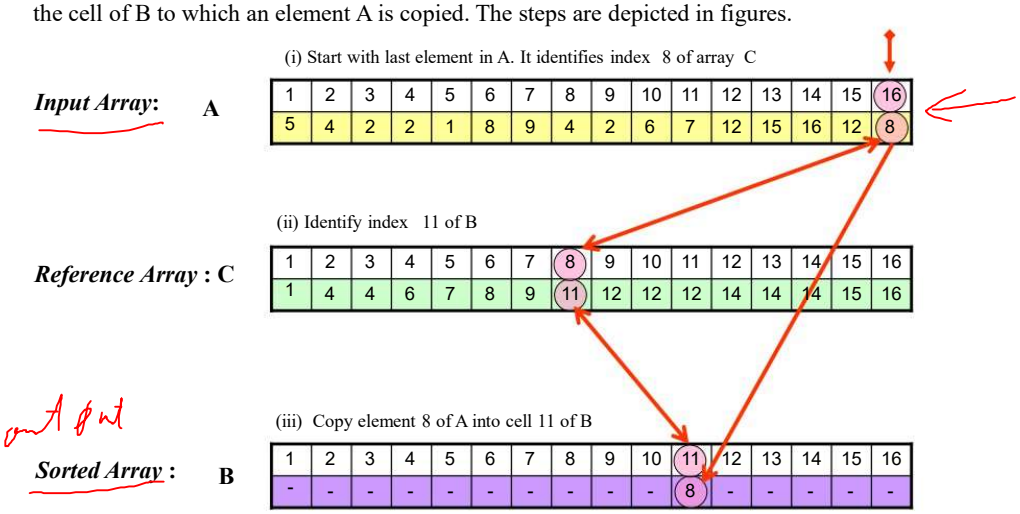
*Step #3: The index **j** is decremented by 1 to access the next left element of array **A***

Steps #2** to **Step #3** are repeated until **j=1

4

Counting Sort-I

The ***input array A*** is sorted into target ***sorted array B***. A ***reference array C*** is used to identify the cell of B to which an element A is copied. The steps are depicted in figures.



5

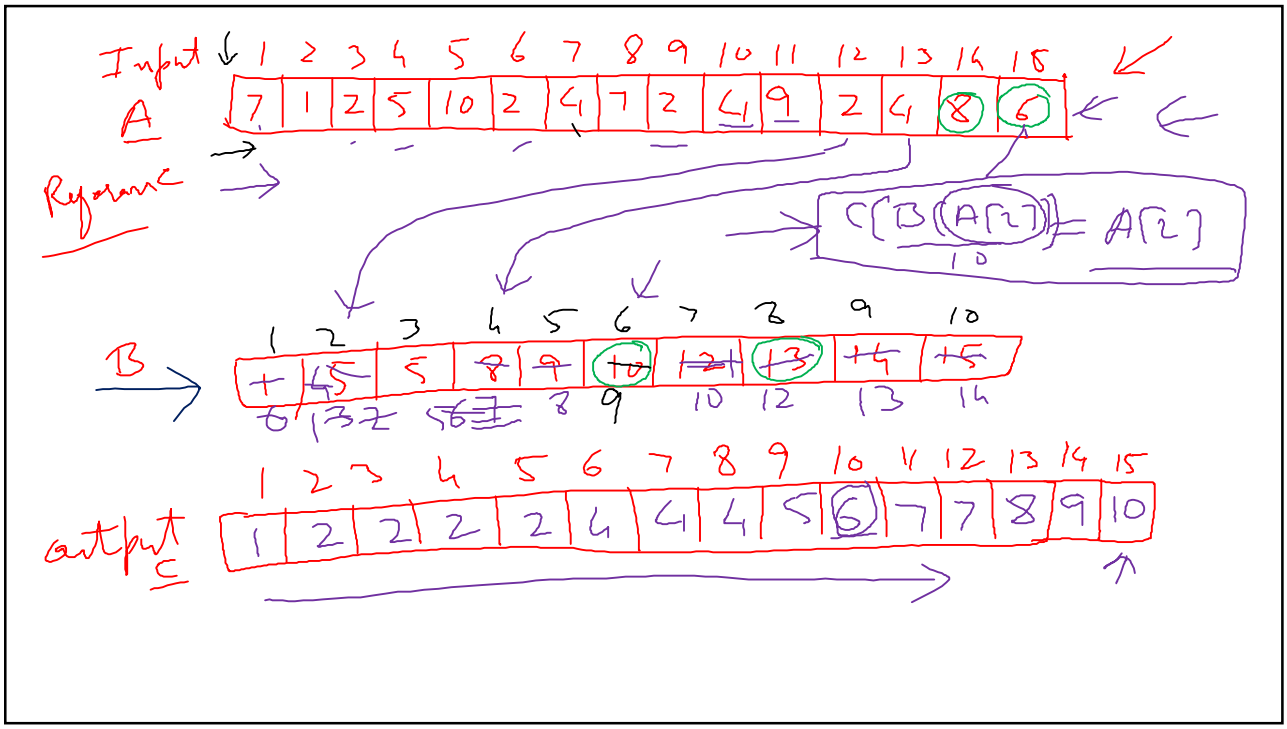
Counting Sort

Pseudo Code

The **COUNTING-SORT** method sorts an ***array of small integers***

```
COUNTING-SORT(A)
1     n=length[A]
2   for j← 0 to n  do          ► initialize the reference array C
3     C[j]← 0
4   for j ← 1 to n do
5       k← A[j]                ► Compute number of elements equal to array index
6     C[k]← C[k] +1
7   for j ← 0 to n  do         ► Compute number of elements equal to or less than index
8     C[j]← C[j] + C[j-1]
9   for j ← n downto 1 do      ► Arrange elements in sorted order
10      i ← A[j]               ► Get index of array C
11      k← C[i]                ► Get Index of array B
12    B[k]← A[j]               ► Copy element of A into B
13    C[i] ← C[i]-1
14  return B                   ► B stores the sorted array
```

6



7

Analysis of Counting Sort

Running Time

The ***Counting Sor***t method uses ***four iterative loops***. Each loop performs *n* iterations. If *a, b, c*, and *d* are unit costs for executing loops then :

1) *Cost of initializing = a.n*

2) *Cost of counting elements equal to indexes = b.n*

3) *Cost of counting elements equal to or less than indexes = c.n*

4) *Cost of accessing indexes and copying elements to auxiliary array = d.n*

Total cost = $a.n + b.n + c.n + d.n = (a + b + c + d).n = \theta(n)$

The running time of Counting Sort is $\theta(n)$

8

Radix Sort

9

Radix Sort

Features

Radix sort algorithm uses the ***composition*** of individual keys to arrange elements in certain order. It can be used to *sort* ***integers, strings of alphabets, ASCII character string*** and ***bits strings***

Sorting algorithm uses a set of ***buckets/bins*** to store intermediate results. The number of buckets depends on ***data type*** of *keys.*

- *The **decimal numbers** need 10 buckets, each corresponding to digits 0-9*
- *The **binary strings** require two buckets, corresponding to bits 0 and 1*
- *The **character strings** consisting of upper case English alphabets would need at least26 buckets*
- *The **ASCII character set** would require 256 buckets.*

10

Radix Sorting of Integers

Algorithm

An array of *positive integers* can sorted using radix sort algorithm. It uses ten buckets to store the results of intermediate processing . The algorithm works as follows:

Step #1: Select the first integer in the array.

Step # 2: Extract the least ***least significant digit*** in the integer.

Step # 3: Store the integer in a bucket identified by the ***digit value***

Step #4: Select next integer in the . Repeat Step # 2 to Step # 3 until all integers have been processed

Step #5: Beginning with the first bucket, ***copy the contents of buckets into the array****.*

Step #6: Select first integer. Extract ***next significant digit****. Repeat Step #3 to Step #5 until digits in all positions up to and including* ***most significant digits*** *have been processed.*

11

Radix Sorting of Integers

Extracting Digits

¾ The ***radix sort of integers*** works by a ***extracting digit*** in specified position in a positive number. The formula for extracting a digit ***k*** in ***p_{th} position*** of an integer ***n*** is

$$k = \left(\lfloor (n / 10_{p-1}) \rfloor \right) \bmod 10$$

For example, if ***n=4 7 6 5 9****, the digit in the* ***third least significant*** *position is extracted by taking p=3.*

$k = (\lfloor 47659 / 10_2 \rfloor) \bmod 10$
$= \lfloor 476.59 \rfloor \bmod 10 = 6$

It follows that the digit in third least significant position of the integer 47659 is 6

The procedure for extracting a digit requires ***fixed*** amount of computation. Thus, it runs in $\theta(1)$ time.

12

Radix Sort

Pseudo Code

The **RADIX-METHOD** sorts a set of positive integers. A digit position is determined by using the procedure **EXTRACT-DIGIT**. A set of queues is used for buckets.

```
RADIX-SORT(A, size)
1  n←length[A]
2  k← size                          ▶maximum number of digits in any key
3    for p←1 to k do                ▶ Cycle through least  to most significant digit in each key
4        for j←1 to n do
5        q ← EXTRACT-DIGIT(A[j],  p)   ▶Extract digit in position p
6     ENQUE( Q[q], A[j])            ▶Store key in the queue identified by  digit position
7        m←1
8     for i← 0 to 9 do
9           while Q[i] is not empty          ▶ Copy until the queue is empty
10                do  A[m]← DEQUE(Q[i])      ▶Copy back to  array
11                    m← m + 1
```

13

Radix Sort Analysis

Running Time

The Radix Sort procedure consists of one outer loop and two inner loops.
The total time for executing the inner and outer loops is worked out as follows:

(1) If d is the ***maximum number of digits*** in any key , then outer loop will be executed d times. Assuming c_1 to be the unit cost, the total cost of executing the outer loop is

$$T_{outer} = c_1.d$$

(2) The first inner loop computes the ***digit position*** and distributes the keys in one of the buckets . If q is the ***number of buckets*** , n the array size , and c_2 the unit cost of computing digit position plus the cost of finding a bucket, then total cost of executing the first loop is given by

$$T_{first} = c_2.\ q.\ n$$

(3) The second inner loop removes the elements from q buckets and copies them into the array . If c_3 is the unit cost of ***finding a bucket and copying***, the total cost of the second loop is

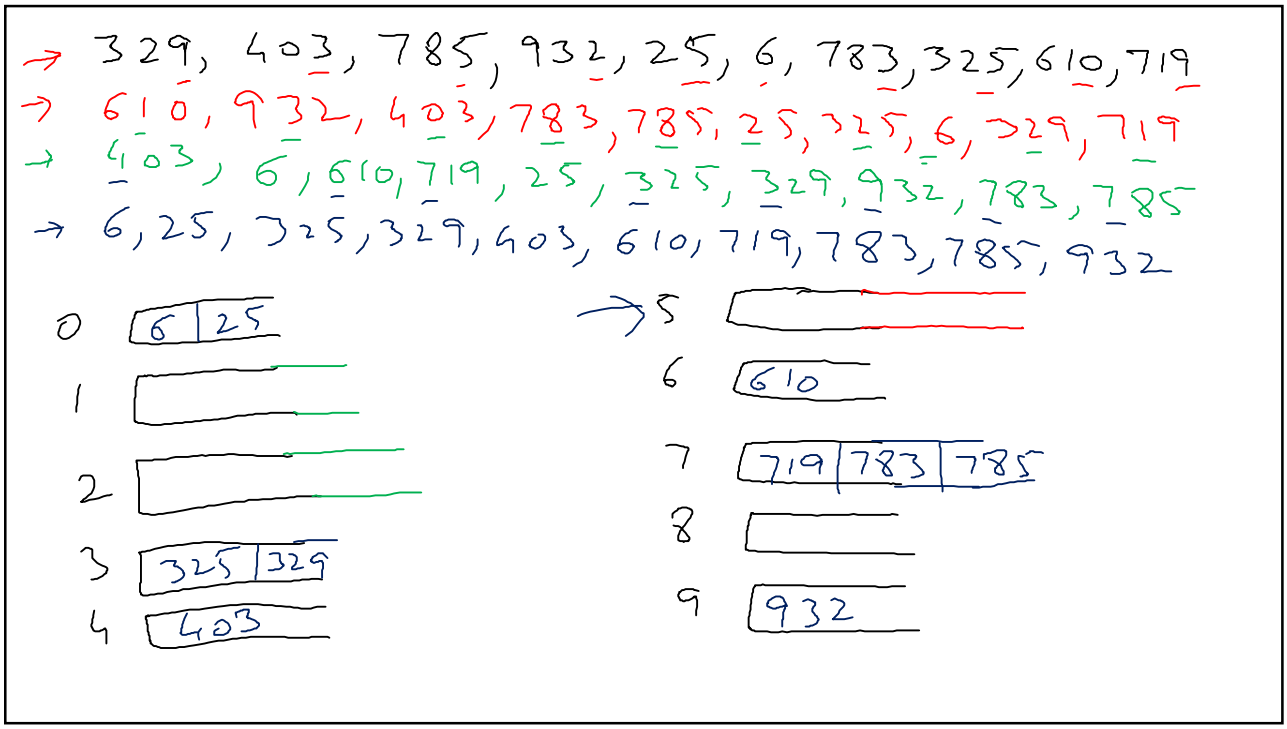
$$T_{second} = c_3.q.n$$

The running time $T(n)$ of radix sort is given by

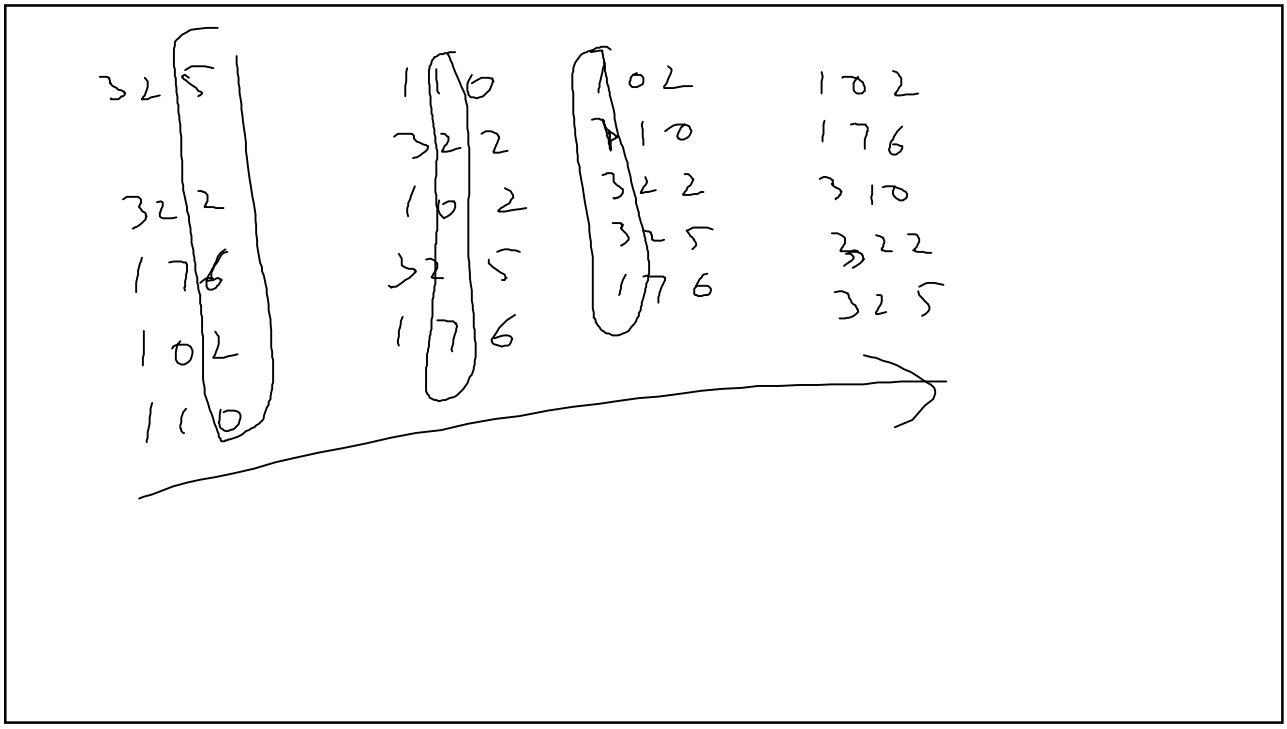
$$T(n)= c_1.d\ (\ c_2\ q.n + c_3.q.n)= c_1(c_2+c_3)d.q.n=\theta(d.q.n)$$

Since d and q are constants, Radix Sort runs in time $\theta(n)$

14


329, 403, 785, 932, 25, 6, 783, 325, 610, 719
610, 932, 403, 783, 785, 25, 325, 6, 329, 719
403, 6, 610, 719, 25, 325, 329, 932, 783, 785
6, 25, 325, 329, 403, 610, 719, 783, 785, 932
0
6
25
1
2
3
325
329
4
403
5
6
610
7
719
783
785
8
9
932

15


325
322
176
102
110
110
322
102
176
102
110
322
325
176
102
110
322
325
176

16